Übungsblatt 2: Speicherhierarchie und Caches

Mikrocomputertechnik 3 · 40 Orientierungspunkte · ca. 75–90 Minuten

Musterlösung

Aufgabe 1: Lokalität an einer Blockgrenze (4 Punkte)

Zwei unabhängige Lesefolgen, jeweils kalter Cache, 32-Byte-Lines, keine Verdrängung:

- A: `0x207C, 0x207C, 0x207C, 0x207C`
- B: `0x2078, 0x207C, 0x2080, 0x2084`

1. Welche Lokalität nutzt jede Folge? (2 P)
2. Geben Sie Hit/Miss je Zugriff und die vollständigen berührten Linebereiche an. Warum garantiert räumliche Lokalität nicht nur einen Miss? (2 P)

Musterlösung 1

A: zeitlich; Miss, Hit, Hit, Hit. B: räumlich; Miss, Hit, Miss, Hit. Linebereiche `0x2060` bis `0x207F` und `0x2080` bis `0x209F`. B überschreitet die Blockgrenze. Je 1 P für die beiden Lokalitäten, 1 P für Folgen und 1 P für Grenzen/Begründung.

Aufgabe 2: AMAT aus einer Messspur im Modell (5 Punkte)

Hitzeit zwei Takte, zusätzliche Misspenalty sechzig Takte. AMAT ist die mittlere Speicherzugriffszeit; alle Anfragen sind abgeschlossen.

1. 120 Anfragen ergeben 108 Hits und zwölf Misses. Bestimmen Sie Missrate und AMAT. Kontrollieren Sie durch Summieren der Einzelzeiten. (3 P)
2. Berechnen Sie AMAT bei 75 Prozent Hitrate. (1 P)
3. Ist das Verhältnis beider Werte bereits ein Programmspeedup? (1 P)

Musterlösung 2

1. $m = 12/120 = 0{,}1$, AMAT $= 2 + 0{,}1 \cdot 60 = 8$ Takte. Kontrolle: $(108 \cdot 2 + 12 \cdot 62)/120 = 960/120 = 8$. (3 P)
2. $2 + 0{,}25 \cdot 60 = 17$ Takte. (1 P)
3. $17/8 = 2{,}125$ vergleicht Zugriffszeiten; übrige Arbeit und Überlappung fehlen. (1 P)

Aufgabe 3: Andere Cachegeometrie (6 Punkte)

32-Bit-Byteadressen, 16 KiB Datenkapazität, 32 Byte je Line; Tags und Zustandsbits sind nicht in der Kapazität enthalten.

1. Bestimmen Sie Sets, Tag-, Index- und Offsetbits für Direct Mapped und 2-Way. (3 P)
2. Zerlegen Sie `0x3A5C91E6` für Direct Mapped und kontrollieren Sie durch Rückrechnung. (2 P)
3. Bestimmen Sie für die Dezimaladresse 1510 Blockbeginn, letztes Blockbyte und Offset. (1 P)

Musterlösung 3

Organisation	Sets	Tagbits	Indexbits	Offsetbits
Direct Mapped	512	18	9	5
2-Way	256	19	8	5

Je 1 P pro Zeile, 1 P: gleicher Offset wegen gleicher Linegröße. Adresse: Offset sechs, Index 143, Tag `0xE972`. Rückrechnung $59762 \cdot 16384 + 143 \cdot 32 + 6 = 979145190 =$ `0x3A5C91E6`. (2 P) Für 1510: Block 47, Beginn 1504, letztes Byte 1535, Offset sechs. (1 P)

Aufgabe 4: Konflikte und Ersatzreihenfolge (6 Punkte)

Cachegeometrie wie Aufgabe 3, beide Caches anfangs leer. Nur diese sieben Reads zählen: `0x3000, 0x7000, 0x3004, 0x7004, 0x3000, 0x7000, 0x3008`.

1. Erstellen Sie eine Tabelle mit DM-Index, DM-Tag und Hit/Miss für Direct Mapped sowie Hit/Miss für 2-Way. Bestimmen Sie beide Quoten und die Ursache zusätzlicher Misses. (4 P)
2. Separates leeres 3-Way-Set, Blockfolge J, K, L, J, M. Geben Sie die Ordnung nach dem zweiten J und das Opfer bei M an: FIFO nach Ladealter, LRU nach letzter Nutzung. (2 P)

Musterlösung 4

Schritt	DM-Index	DM-Tag	DM	2-Way
1	384	0	Miss	Miss
2	384	1	Miss	Miss
3	384	0	Miss	Hit
4	384	1	Miss	Hit
5	384	0	Miss	Hit
6	384	1	Miss	Hit
7	384	0	Miss	Hit

DM: null Hits, sieben Misses. 2-Way: fünf Hits, zwei Misses, $5/7 \approx 71{,}43\,\%$. Dort Set 128, Tags eins und drei. Die zusätzlichen DM-Misses sind Konflikte: Zwei Blöcke passen in die Gesamtkapazität. (4 P) FIFO: J, K, L; Opfer J. LRU: K, L, J; Opfer K. Nur LRU verändert beim Hit die Nutzungsordnung. (2 P)

Aufgabe 5: Writeback tritt vor dem letzten Zugriff auf (5 Punkte)

Direct Mapped wie Aufgabe 3, Write-Back und Write-Allocate, keine Puffer. Anfangs ist Block `0x3000` gültig und sauber; sein betrachtetes Wort ist zwei, auch in der unteren Ebene. Block `0x7000` fehlt im Cache und enthält unten acht.

Folge: Store sechs nach `0x3000`; Load `0x7000`; Load `0x3000`; Store elf nach `0x3000`.

1. Notieren Sie nach jedem Schritt Blockbasis, Wortwert, Dirty, unteren Wert bei `0x3000` und eine mögliche Writeback-Adresse. (3 P)
2. Aus welchen alten Informationen entsteht die Writeback-Adresse? (1 P)
3. Wie wirkt No-Write-Allocate bei einem Store-Miss? Folgt die Regel aus Write-Back? (1 P)

Musterlösung 5

Schritt	Blockbasis	Wert	Dirty	Unten bei 0x3000	Writeback
Store 6	`0x3000`	6	1	2	keiner
Load 0x7000	`0x7000`	8	0	6	`0x3000`
Load 0x3000	`0x3000`	6	0	6	keiner
Store 11	`0x3000`	11	1	6	keiner

Tabelle 3 P. Alte Basis $(0 \cdot 512 + 384) \cdot 32 =$ `0x3000` aus altem Tag und Index, nicht aus der neu angefragten Adresse. (1 P) No-Write-Allocate gibt den Store weiter, ohne eine neue Line zuzuteilen; diese Entscheidung ist von Write-Back getrennt. (1 P)

Aufgabe 6: Ein rechteckiges C-Array (6 Punkte)

`uint32_t feld[12][8]`, vier Byte je Element, ausgerichtete Basis `0x3000`. Zwei unabhängige Modellspuren laden genau einmal jedes Element: A zeilenweise, B spaltenweise. Keine anderen Zugriffe oder Optimierungen.

1. Geben Sie die Adressformel und die ersten vier Adressen beider Spuren an. (2 P)
2. Direct Mapped, 128 Byte Datenkapazität, 32-Byte-Lines, vor jeder Spur kalt: Bestimmen Sie Set-Index und vollständige Hit/Miss-Bilanzen. (3 P)
3. Wie ändert sich B bei 512 Byte Kapazität und sonst gleichen Bedingungen? (1 P)

Musterlösung 6

1. $a_{r,c} = \texttt{0x3000} + 4(8r + c)$. A: `3000, 3004, 3008, 300C`; B: `3000, 3020, 3040, 3060`, jeweils hexadezimal. (2 P)
2. Vier Sets, eine Arrayzeile je Line; Index $(384 + r) \bmod 4 = r \bmod 4$. A: zwölf Misses, 84 Hits. B: 96 Misses, null Hits; je Spalte verdrängen zwölf Zeilen einander in vier Sets. (3 P)
3. Sechzehn Sets halten alle zwölf Zeilen: B hat zwölf Misses und 84 Hits. (1 P)

Aufgabe 7: Ein Cache-Controller auf Papier (8 Punkte)

16-Bit-Byteadressen, 32-Bit-Wörter. 2-Way-Cache mit vier Sets und 16 Byte je Line, Write-Back, Write-Allocate, LRU. Ein ungültiger Weg wird vor einem gültigen belegt. Bei einem Miss schreibt der Controller ein schmutziges Opfer zuerst zurück, lädt dann den ganzen Block und führt erst danach Load oder Store aus. Datenwerte sind dezimal, Adressen und Tags hexadezimal.

Cache, Ausschnitt Set 1 und 2. In Set 1 ist Weg 0 am längsten unbenutzt.

Set	Weg	V	D	Tag	Wort 0	Wort 1	Wort 2	Wort 3
1	0	1	0	`0x048`	10	11	12	13
1	1	1	1	`0x091`	40	41	99	43
2	0	1	0	`0x0E9`	70	71	72	73
2	1	0	0	`0x07D`	5	6	7	8

DRAM, Ausschnitt. Jede Zeile ist ein ausgerichteter 16-Byte-Block.

Blockadresse	`+0x0`	`+0x4`	`+0x8`	`+0xC`
`0x0D10`	30	31	32	33
`0x1210`	10	11	12	13
`0x1F60`	60	61	62	63
`0x2450`	40	41	42	43
`0x3210`	50	51	52	53
`0x3A60`	70	71	72	73

1. Bestimmen Sie Offset-, Index- und Tagbits. Zerlegen Sie `0x2458`, `0x1F64`, `0x0D1C` und `0x3214` in Tag, Index, Offset und Wortnummer. Berechnen Sie aus Tag und Index die Blockadressen der gültigen Cache-Lines. (2 P)
2. Führen Sie den Controller für die Zugriffe in Programmreihenfolge aus. LRU danach ist der Weg, den das betroffene Set als nächsten verdrängt. (4 P)
3. Geben Sie den Endzustand von Set 1 und 2 sowie alle geänderten DRAM-Wörter an. Welches DRAM-Wort ist am Ende veraltet? Warum ist das bei Write-Back zulässig? (2 P)

Nr.	Zugriff	Hit/Miss	Weg	Writeback	Nachgeladen	Wert	LRU danach
1	Load `0x2458`						
2	Load `0x1F64`						
3	Store 77 nach `0x0D1C`						
4	Load `0x3214`						
5	Load `0x0D1C`						
6	Load `0x2458`						

Musterlösung 7

Teil 1: Offset 4 Bit, Index 2 Bit, Tag $16 - 6 = 10$ Bit; Tag = Adresse $\gg 6$. (1 P)

Adresse	Tag, Index, Offset binär	Tag	Index	Offset (Wort)
`0x2458`	`0010010001 01 1000`	`0x091`	1	8 (2)
`0x1F64`	`0001111101 10 0100`	`0x07D`	2	4 (1)
`0x0D1C`	`0000110100 01 1100`	`0x034`	1	12 (3)
`0x3214`	`0011001000 01 0100`	`0x0C8`	1	4 (1)

Blockadresse = Tag $\cdot 64 +$ Index $\cdot 16$: Set 1 `0x1210` und `0x2450`, Set 2 Weg 0 `0x3A60`. Set 2 Weg 1 ist ungültig. (1 P)

Teil 2:

Nr.	Zugriff	Hit/Miss	Weg	Writeback	Nachgeladen	Wert	LRU danach
1	Load `0x2458`	Hit	1	–	–	99	0
2	Load `0x1F64`	Miss	1	–	`0x1F60`	61	0
3	Store 77 nach `0x0D1C`	Miss	0	–	`0x0D10`	77	1
4	Load `0x3214`	Miss	1	`0x2450`	`0x3210`	51	0
5	Load `0x0D1C`	Hit	0	–	–	77	1
6	Load `0x2458`	Miss	1	–	`0x2450`	99	0

Zwei Hits, vier Misses, ein Writeback. Tabelle 3 P. Zugriff 1 liefert 99 aus der schmutzigen Line, nicht 42 aus dem DRAM. Zugriff 2 trifft den passenden Tag bei V = 0: Miss, die Werte 5 bis 8 sind bedeutungslos. Zugriff 3 lädt den sauberen LRU-Weg 0 neu, schreibt 77 in Wort 3 und setzt D = 1. Zugriff 4 verdrängt den schmutzigen Weg 1; `0x2450` folgt aus altem Tag `0x091` und Index 1. Zugriff 6 lädt den zurückgeschriebenen Block und erhält wieder 99. (1 P)

Teil 3:

Set	Weg	V	D	Tag	Wort 0	Wort 1	Wort 2	Wort 3
1	0	1	1	`0x034`	30	31	32	77
1	1	1	0	`0x091`	40	41	99	43
2	0	1	0	`0x0E9`	70	71	72	73
2	1	1	0	`0x07D`	60	61	62	63

In beiden Sets ist danach Weg 0 LRU. Im DRAM ändert sich nur `0x2458` von 42 auf 99. Veraltet ist `0x0D1C` mit 33, der Cache hält dort 77 mit D = 1. Write-Back schreibt erst beim Verdrängen zurück; zulässig, solange alle Zugriffe über diesen Cache laufen. (2 P)

Optional: Aufgabe 6 im Cache-Simulator Ripes

Das folgende RV32I-Programm erzeugt die beiden Spuren aus Aufgabe 6. Die einzigen Datenzugriffe sind die 96 `lw`; `feld` liegt am Beginn des `.data`-Segments, in Ripes standardmäßig `0x10000000` statt `0x3000`.

```
# Ripes, RV32I: Spuren aus Aufgabe 6, uint32_t feld[12][8]
# MODUS 0 = Spur A (zeilenweise), MODUS 1 = Spur B (spaltenweise)

.data
feld:   .zero 384           # 12 * 8 Elemente * 4 Byte

.text
main:
        li   t0, 0          # MODUS hier umstellen: 0 = A, 1 = B
        la   s0, feld       # erstes Element der Außenschleife
        bnez t0, spur_b
        li   s1, 12         # A: außen 12 Zeilen
        li   s2, 8          #    innen 8 Spalten
        li   s3, 32         #    außen +32 Byte: nächste Zeile
        li   s4, 4          #    innen +4 Byte: nächstes Element
        j    start
spur_b:
        li   s1, 8          # B: außen 8 Spalten
        li   s2, 12         #    innen 12 Zeilen
        li   s3, 4          #    außen +4 Byte: nächste Spalte
        li   s4, 32         #    innen +32 Byte: nächste Zeile
start:
        li   a0, 0          # Summe aller Elemente
aussen:
        mv   t1, s0         # Adresse des ersten Elements
        mv   t2, s2         # Zähler der Innenschleife
innen:
        lw   t3, 0(t1)      # einziger Datenzugriff
        add  a0, a0, t3
        add  t1, t1, s4
        addi t2, t2, -1
        bnez t2, innen
        add  s0, s0, s3
        addi s1, s1, -1
        bnez s1, aussen
        li   a7, 10         # Ecall exit: Programm beenden
        ecall
```

Cache-Simulation. Ripes (Desktop oder ripes.dk), Prozessor *Single-cycle processor*, RISC-V 32 Bit: Pipeline-Modelle können beim Anhalten zusätzliche Zugriffe zählen. Ausgewertet wird nur der Datencache *L1 Data Cache*; der Befehlscache *L1 Instr. Cache* zählt die Befehlsabrufe. Ripes erwartet Sets, Ways und Words per Line als Exponent N von 2^N; ein Wort hat vier Byte. Ersetzung LRU, Wr. hit *Write-back*, Wr. miss *Write allocate* (ohne Wirkung, da das Programm nicht schreibt). Vor jedem Lauf MODUS setzen, Cache einstellen und *Reset* drücken; die Zähler stehen unter der Cache-Ansicht.

Lauf	MODUS	N: Sets, Ways, Words	Cache	Prognose: Hits, Misses	Ripes: Reads, Hits, Misses
1	0 = A	2, 0, 3	128 Byte, Direct Mapped		
2	1 = B	2, 0, 3	128 Byte, Direct Mapped		
3	1 = B	4, 0, 3	512 Byte, Direct Mapped		
4	1 = B	1, 1, 3	128 Byte, 2-Way, LRU		

1. Tragen Sie vor jedem Lauf Ihre Prognose aus Aufgabe 6 ein, danach die Ripes-Zähler. Warum zeigt der Datencache genau 96 Reads?
2. Lauf 2 im Einzeltakt (*Clock*): Notieren Sie für die ersten fünf Loads Adresse, Set, Tag und Hit/Miss. Welche Arrayzeile verdrängt Zeile 0? Warum trifft der erste Load der zweiten Spalte Zeile 0 nicht mehr an?
3. Lauf 4 hat die Kapazität von Lauf 2, aber zwei Wege. Erklären Sie sein Ergebnis im Vergleich zu Lauf 3: Konflikt oder fehlende Kapazität?
4. Die Set-Indizes stimmen mit Aufgabe 6 überein, die Tags nicht. Warum?

Musterlösung (optional)

Lauf	Reads	Hits	Misses	Hitrate
1	96	84	12	0,875
2	96	0	96	0
3	96	84	12	0,875
4	96	0	96	0

1. Nur die `lw` greifen auf Daten zu; `la` und `li` lesen keinen Speicher, Befehlsabrufe zählt der Befehlscache. Writes und Writebacks bleiben null.
2. `0x10000000`, `0x10000020`, `0x10000040`, `0x10000060`, `0x10000080`; Sets 0, 1, 2, 3, 0; Tag (Adresse $\gg 7$) `0x200000` für Zeilen 0–3, `0x200001` für Zeile 4; alle Miss. Zeile 4 verdrängt Zeile 0 aus Set 0, danach belegt Zeile 8 dieses Set. Spalte 1 beginnt wieder bei Zeile 0: Miss.
3. Eine Spalte berührt zwölf Lines, 128 Byte halten nur vier. Im 2-Way-Cache wechseln sich je Set sechs Zeilen auf zwei Plätzen ab; LRU verdrängt jede Zeile, bevor sie wiederkommt. Es fehlt Kapazität, nicht Assoziativität. 512 Byte halten alle zwölf Zeilen: nur Pflicht-Misses.
4. `0x3000` und `0x10000000` sind Vielfache von 512. Die unteren neun Adressbits mit Offset und Index sind daher gleich, nur die Tagbits unterscheiden sich, bei 128 Byte etwa `0x60` statt `0x200000` für Zeile 0.